

Alert Processing Middleware - Technical Documentation

1. Executive Summary

The Wazuh Alert Processing Middleware is a Java-based distributed system designed to manage security alerts from Wazuh SIEM. The system provides flexible alert processing capabilities through multiple deployment modes, supporting both batch processing and real-time streaming architectures.

Key Capabilities

- **Query & Export:** Filter alerts by severity and export to JSON files and RabbitMQ
- **Multi-threaded Processing:** Concurrent handling of file ingestion, database operations, and message queue consumption
- **Dual Processing Modes:** Support for both batch file processing and real-time message streaming
- **Bidirectional Data Flow:** Can both publish alerts to RabbitMQ and consume alerts from RabbitMQ
- **Fault Tolerance:** Graceful degradation when external services are unavailable

Use Cases

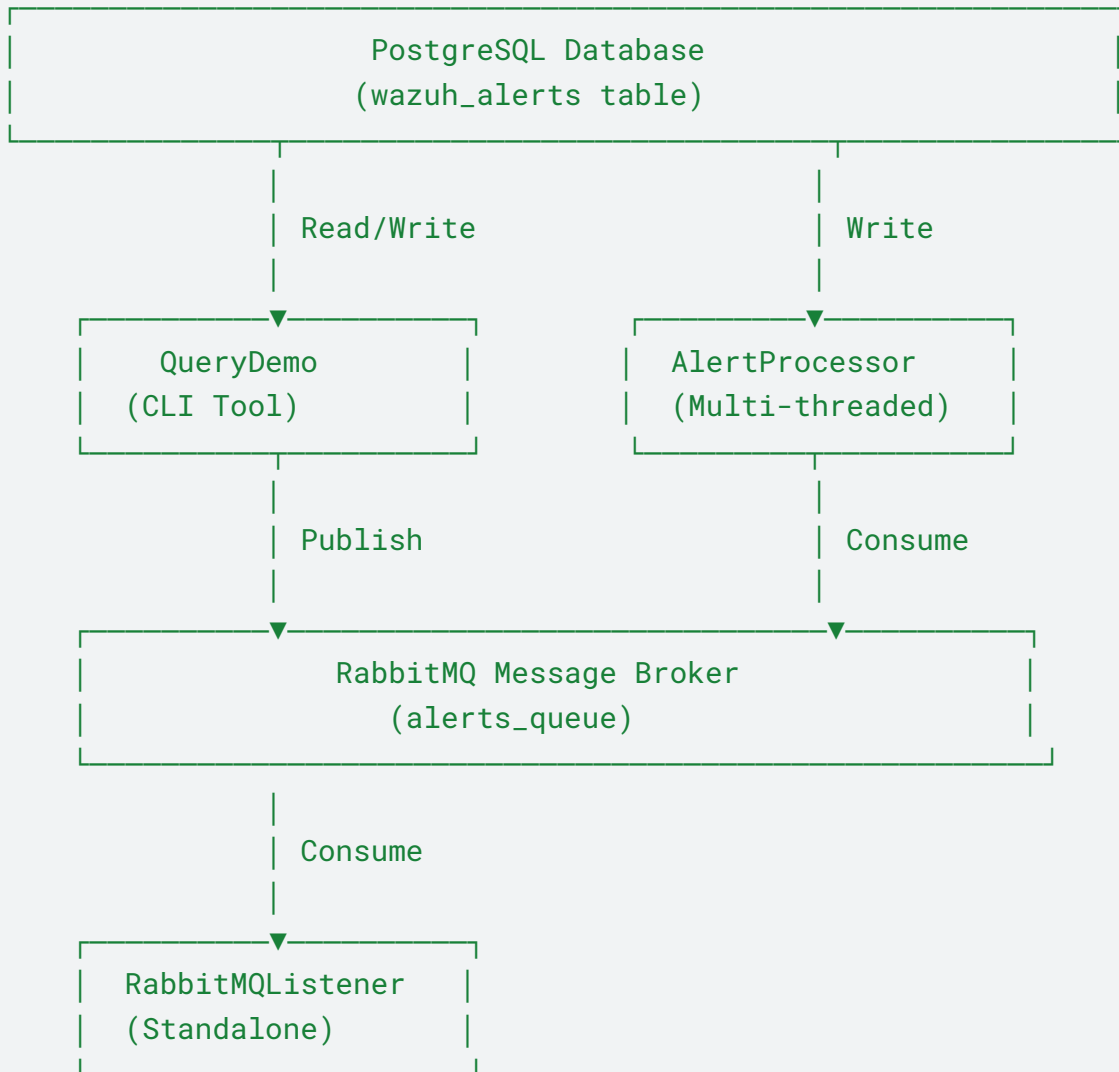
- **Alert Distribution:** Export filtered alerts to downstream systems via RabbitMQ
 - **Alert Ingestion:** Consume alerts from external sources and persist to PostgreSQL
 - **Alert Analysis:** Query historical alerts by severity for reporting and analysis
 - **System Integration:** Bridge between Wazuh deployments and custom security workflows
-

2. System Architecture

High-Level Architecture

The system employs a modular architecture with clear separation of concerns:

None



Technology Stack

- **Language:** Java (SE 8+)
- **Database:** PostgreSQL with JSONB support
- **Message Broker:** RabbitMQ 3.x
- **Key Libraries:**
 - `com.rabbitmq:amqp-client` - RabbitMQ client
 - `org.postgresql:postgresql` - JDBC driver
 - `org.json:json` - JSON processing

Component Relationships

1. **WazuhAlertDao**: Central data access layer used by all components
 2. **QueryDemo**: Reads from database, publishes to RabbitMQ and files
 3. **AlertProcessor**: Reads from files and RabbitMQ, writes to database
 4. **RabbitMQListener**: Dedicated consumer, writes to database
 5. **DatabaseUtil**: Utility for connection management (alternative configuration)
-

3. Core Components

3.1 WazuhAlertDao (Data Access Object)

Purpose: Provides database abstraction for alert operations.

Key Methods:

- `findBySeverity(String severity)`: Queries alerts filtered by JSONB payload severity field
- `insertAlert(JSONObject alert)`: Inserts or ignores (ON CONFLICT) alert records

Database Schema:

SQL

```
Table: public.wazuh_alerts
- event_id (UUID, PRIMARY KEY)
- timestamp (TIMESTAMP)
- event_type (VARCHAR)
- source_module (VARCHAR)
- payload (JSONB)
```

Features:

- Direct JSONB querying using PostgreSQL's `->>` operator
- UUID generation for missing event_id values
- Timestamp parsing with fallback to current time
- Conflict resolution via `ON CONFLICT DO NOTHING`

3.2 QueryDemo (Query & Export Tool)

Purpose: Command-line tool for filtering alerts and distributing them to multiple outputs.

Execution:

Shell

```
java -cp ... QueryDemo <severity>
```

Workflow:

1. Accepts severity parameter from command line
2. Queries database using `WazuhAlertDao.findBySeverity()`
3. Creates `output/` directory if not exists
4. Attempts RabbitMQ connection (continues if unavailable)
5. For each alert:
 - Writes JSON file: `output/alert_N.json` (pretty-printed)
 - Publishes to RabbitMQ: `alerts_queue` (if connected)
6. Reports export statistics

Configuration:

- RabbitMQ Host: `192.168.86.76:5672`
- Credentials: `guest:guest`
- Queue: `alerts_queue` (durable)

3.3 AlertProcessor (Multi-threaded Engine)

Purpose: Concurrent processing engine with three dedicated threads.

Architecture:

None

```
ExecutorService (FixedThreadPool: 3 threads)
├─ Thread 1: DatabaseConnectionPool (Health Monitor)
├─ Thread 2: FileProcessor (Batch Ingestion)
└─ Thread 3: RabbitMQListenerThread (Real-time Consumer)
```

Thread Details:

Thread 1: DatabaseConnectionPool

- Maintains database connection health
- Provides shared `WazuhAlertDao` instance
- Monitors connection status every 5 seconds

Thread 2: FileProcessor

- Scans `messages/` directory for JSON files at startup
- Processes files in sorted order
- Flattens nested payload structure
- Inserts alerts via shared DAO
- Reports success/failure statistics

Thread 3: RabbitMQListenerThread

- Connects to RabbitMQ on startup
- Consumes messages from `alerts_queue`
- Processes messages with manual acknowledgment
- Flattens payload and inserts to database
- Handles errors with negative acknowledgment (requeue)

Lifecycle Management:

- Graceful shutdown via shutdown hook
- 10-second timeout for thread termination
- Proper resource cleanup (channels, connections)

3.4 RabbitMQListener (Standalone Consumer)

Purpose: Dedicated, single-purpose RabbitMQ consumer.

Differences from AlertProcessor:

- Single-threaded, focused on message consumption
- No file processing capability
- Simpler deployment model
- Uses `wait()` mechanism to keep alive

Features:

- QoS prefetch limit: 1 message at a time
- Manual message acknowledgment
- Automatic requeue on processing failure
- Graceful shutdown support

4. Message Flow & Triggers

4.1 Database-to-RabbitMQ Flow (QueryDemo)

Trigger: Manual command-line execution with severity parameter.

Process Flow:

None

User executes:

```
java QueryDemo "high"
```

|

▼

Query Database

```
SELECT * WHERE payload->'severity' = 'high'
```

|

▼

For each alert:

└─ Write to file: output/alert_N.json

└─ Publish to RabbitMQ: alerts_queue

|

▼

Report: "✅ Exported 15 alerts with severity = high"

Trigger Characteristics:

- **Manual:** Requires explicit user invocation
- **On-demand:** Not scheduled or automated
- **Filtered:** Only alerts matching severity criteria are published
- **Dual Output:** Simultaneously writes to filesystem and message queue

Message Format: Messages published to RabbitMQ contain:

JSON

```
{
  "event_id": "uuid-string",
  "timestamp": "2025-10-07T10:30:00Z",
  "event_type": "security_event",
  "source_module": "wazuh",
  "payload": {
    "severity": "high",
    "rule_id": "5502",
```

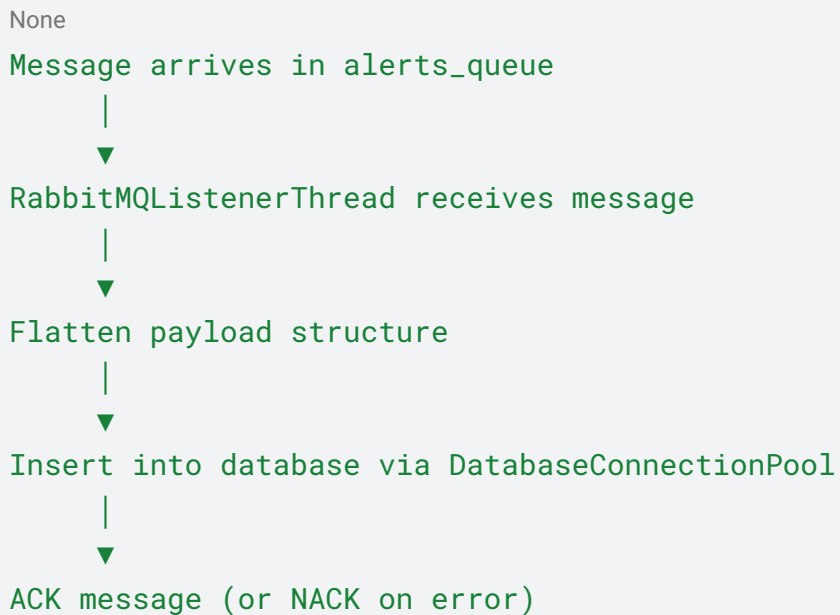
```
    "description": "..."  
  }  
}
```

4.2 RabbitMQ-to-Database Flow (Consumers)

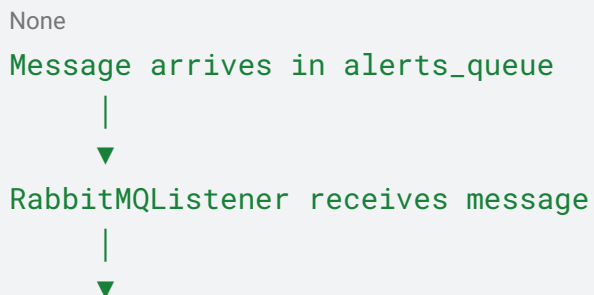
Trigger: Arrival of new messages in `alerts_queue`.

Consumer Options:

Option A: AlertProcessor Thread 3



Option B: RabbitMQListener



```
Flatten payload structure
|
▼
Insert into database via WazuhAlertDao
|
▼
ACK message (or NACK on error)
```

Trigger Characteristics:

- **Event-driven:** Automatically triggered by message arrival
- **Real-time:** Immediate processing upon message receipt
- **Continuous:** Runs indefinitely until shutdown
- **Reliable:** Uses acknowledgments to ensure message processing

4.3 File-to-Database Flow (AlertProcessor Thread 2)

Trigger: Application startup (FileProcessor thread).

Process Flow:

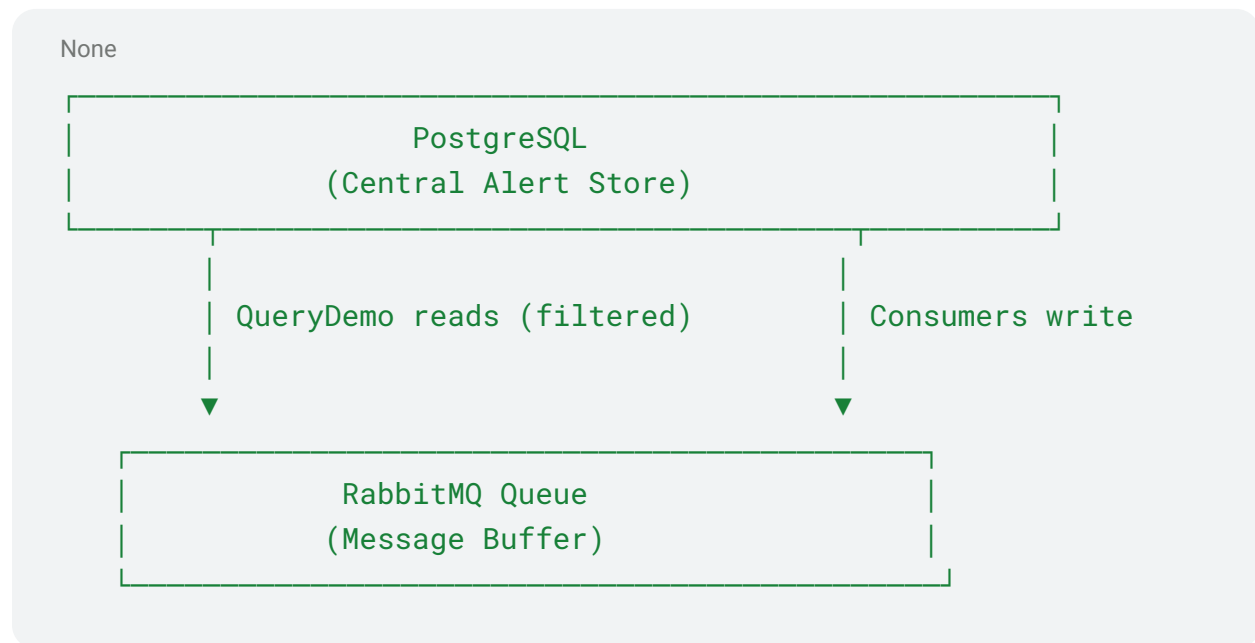
```
None
AlertProcessor starts
|
▼
FileProcessor scans messages/ directory
|
▼
For each *.json file (sorted):
  ↳ Read file content
  ↳ Parse JSON
  ↳ Flatten payload
  ↳ Insert into database
|
▼
Report: "📊 Complete! Success: 10, Failed: 0"
```


Trigger Characteristics:

- **Startup-based:** Executes once when AlertProcessor starts
- **Batch-oriented:** Processes all files in directory
- **Ordered:** Files processed in alphabetical order
- **One-time:** Does not continuously monitor for new files

4.4 Bidirectional Data Flow Pattern

The system supports a complete bidirectional flow:



Use Case Example:

1. External system publishes alerts to RabbitMQ
2. AlertProcessor consumes and stores in PostgreSQL
3. Administrator runs QueryDemo to filter high-severity alerts
4. QueryDemo publishes filtered alerts back to RabbitMQ
5. Downstream systems consume filtered alerts for action

4.5 Payload Flattening Mechanism

Both consumers implement payload flattening to normalize data structure:

Before Flattening:

JSON

```
{
  "event_id": "123",
  "timestamp": "2025-10-07T10:30:00Z",
  "payload": {
    "severity": "high",
    "rule_id": "5502"
  }
}
```

After Flattening:

JSON

```
{
  "event_id": "123",
  "timestamp": "2025-10-07T10:30:00Z",
  "severity": "high",
  "rule_id": "5502"
}
```

This ensures compatibility with the database schema where `payload` is a JSONB field but top-level fields are also extracted.

5. Data Flow & Processing Pipeline

5.1 Batch Processing Pipeline (File-based)

Input: JSON files in `messages/` directory

Steps:

1. Directory scan and file discovery
2. Alphabetical sorting
3. Sequential file reading
4. JSON parsing and validation
5. Payload flattening

6. Database insertion with conflict handling
7. Statistics reporting

Characteristics:

- **Synchronous:** Processes one file at a time
- **Startup-only:** Runs once per application start
- **Volume-oriented:** Suitable for bulk historical data import

5.2 Real-time Processing Pipeline (Queue-based)

Input: Messages from RabbitMQ `alerts_queue`

Steps:

1. Message receipt from queue
2. UTF-8 decoding
3. JSON parsing
4. Payload flattening
5. Database insertion
6. Message acknowledgment
7. Console logging

Characteristics:

- **Asynchronous:** Event-driven processing
- **Continuous:** Runs indefinitely
- **Low-latency:** Immediate processing upon receipt
- **Reliable:** Manual ACK/NACK ensures delivery guarantees

5.3 Export Pipeline (Query-based)

Input: Database query results

Steps:

1. Severity-based query execution
2. Result set iteration
3. JSON pretty-printing to file
4. RabbitMQ message publishing
5. Statistics aggregation
6. Summary reporting

Characteristics:

- **On-demand:** Triggered by user action

- **Filtered:** Only matching records processed
- **Dual-output:** File and queue simultaneously

5.4 Database Schema Considerations

Alert Storage Strategy:

- **Structured fields:** event_id, timestamp, event_type, source_module for indexing
- **Flexible payload:** JSONB field for arbitrary alert metadata
- **Deduplication:** ON CONFLICT ensures idempotency

Query Performance:

- JSONB operator `->>` enables efficient JSON field queries
- Index recommendations:

SQL

```
CREATE INDEX idx_severity ON wazuh_alerts
((payload->>'severity'));CREATE INDEX idx_timestamp ON
wazuh_alerts (timestamp DESC);
```

6. Threading Model

6.1 Concurrent Execution Strategy

The AlertProcessor uses a **fixed thread pool** pattern with three dedicated worker threads:

Java

```
ExecutorService executorService =
Executors.newFixedThreadPool(3);
```

Design Rationale:

- **Fixed Size:** Prevents thread explosion under load
- **Dedicated Threads:** Each thread has a single, well-defined responsibility
- **Shared Resource:** All threads access database via shared DAO instance

6.2 Thread Responsibilities

Thread	Purpose	Lifecycle	Blocking Operations
DB-Pool	Connection health monitoring	Periodic sleep (5s)	Sleep only
File-Sender	Batch file ingestion	Startup, then terminates	File I/O, DB writes
RabbitMQ-Listener	Queue message consumption	Continuous until shutdown	Network I/O, DB writes

6.3 Thread Lifecycle

Startup Sequence:

1. Main thread creates AlertProcessor instance
2. ExecutorService initialized with 3 threads
3. Three tasks submitted simultaneously
4. Each thread sets its name for debugging
5. Main thread registers shutdown hook
6. Main thread enters wait state (`join()`)

Shutdown Sequence:

1. SIGTERM/SIGINT received
2. Shutdown hook invoked
3. Threads stop gracefully:
 - RabbitMQListener closes channels/connections
 - FileProcessor stops processing (already completed)
 - DatabaseConnectionPool stops health monitoring
4. ExecutorService.shutdown() called
5. 10-second grace period for termination
6. Force shutdown if timeout exceeded

6.4 Synchronization & Safety

Thread-Safe Components:

- `volatile boolean running`: Ensures visibility across threads
- `WazuhAlertDao`: Stateless, creates new connections per operation
- `DatabaseConnectionPool`: Provides thread-safe DAO access

Potential Considerations:

- No explicit synchronization needed (operations are independent)
- Database handles concurrent writes via ACID properties
- RabbitMQ handles concurrent consumers via prefetch limits

Race Condition Prevention:

- No shared mutable state between threads
 - Each thread operates on distinct data sources
 - Database deduplication via `ON CONFLICT` handles concurrent inserts
-

7. Integration Points

7.1 PostgreSQL Database Integration

Connection Parameters:

- **Host:** `192.168.86.28:5432`
- **Database:** `wazuhdb`
- **User:** `postgres`
- **Password:** `postgres`
- **Driver:** `org.postgresql.Driver`

Connection Strategy:

- **Per-operation connections:** New connection for each DAO method
- **No connection pooling:** Simple approach, suitable for moderate load
- **Alternative:** DatabaseUtil provides different connection configuration

JSONB Integration:

```
Java
// Query JSONB field
WHERE payload->>'severity' = ?

// Insert JSONB field
stmt.setObject(5, jsonString, java.sql.Types.OTHER);
```

Schema Requirements:

- Table: `public.wazuh_alerts`

- UNIQUE constraint on `event_id` for conflict resolution
- JSONB support (PostgreSQL 9.4+)

7.2 RabbitMQ Message Broker Integration

Connection Parameters:

- **Host:** `192.168.86.76:5672`
- **Virtual Host:** `/` (default)
- **Username:** `guest`
- **Password:** `guest`

Queue Configuration:

- **Name:** `alerts_queue`
- **Durable:** `true` (survives broker restart)
- **Exclusive:** `false` (multiple consumers allowed)
- **Auto-delete:** `false` (persists when consumers disconnect)

Producer Configuration (QueryDemo):

Java

```
channel.queueDeclare(QUEUE_NAME, true, false, false, null);
channel.basicPublish("", QUEUE_NAME, null,
message.getBytes("UTF-8"));
```

Consumer Configuration:

Java

```
channel.basicQos(1); // Prefetch limit
channel.basicConsume(QUEUE_NAME, false, deliverCallback,
cancelCallback);
```

Delivery Guarantees:

- **Publisher:** Fire-and-forget (no confirms)
- **Consumer:** Manual ACK ensures at-least-once delivery
- **Requeue:** NACK with `requeue=true` on processing failure

7.3 File System Interactions

Input Directory: `messages/`

- Purpose: Batch ingestion source
- File pattern: `*.json`
- Processing: One-time scan at startup
- Created automatically if missing

Output Directory: `output/`

- Purpose: Query export destination
- File naming: `alert_N.json` (sequential)
- Format: Pretty-printed JSON (4-space indent)
- Created automatically if missing

File Operations:

- Read: `Files.readString()` (Java 11+)
 - Write: `FileWriter` with try-with-resources
 - Error handling: Continue processing on individual file failures
-

8. Deployment & Configuration

8.1 Prerequisites

Required Software:

- Java Development Kit (JDK) 8 or higher
- PostgreSQL 9.4+ with JSONB support
- RabbitMQ 3.x server
- Maven or Gradle for dependency management

Required Dependencies:

XML

```
<dependencies>
  <dependency>
    <groupId>com.rabbitmq</groupId>
    <artifactId>amqp-client</artifactId>
    <version>5.x.x</version>
```



```
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.x.x</version>
</dependency>
<dependency>
  <groupId>org.json</groupId>
  <artifactId>json</artifactId>
  <version>20230227</version>
</dependency>
</dependencies>
```

8.2 Database Setup

Create Database:

SQL

```
CREATE DATABASE wazuhdb;
```

Create Table:

SQL

```
CREATE TABLE public.wazuh_alerts (
  event_id UUID PRIMARY KEY,
  timestamp TIMESTAMP NOT NULL,
  event_type VARCHAR(100),
  source_module VARCHAR(100),
  payload JSONB
);
```

Recommended Indexes:

SQL

```
CREATE INDEX idx_timestamp ON wazuh_alerts (timestamp DESC);  
CREATE INDEX idx_severity ON wazuh_alerts  
((payload->>'severity'));  
CREATE INDEX idx_rule_id ON wazuh_alerts ((payload->>'rule_id'));
```

8.3 RabbitMQ Setup

Create Queue (or let application auto-create):

Shell

```
rabbitmqctl add_vhost /  
rabbitmqctl set_permissions -p / guest ".*" ".*" ".*"
```

Queue will be auto-declared by the application with:

- Name: `alerts_queue`
- Durable: Yes
- Auto-delete: No

8.4 Running the Components

Option 1: Query & Export Tool

Shell

```
# Compile  
javac -cp "lib/*" com/yourorg/middleware/QueryDemo.java  
  
# Run with severity filter  
java -cp "lib/*" com.yourorg.middleware.QueryDemo high  
java -cp "lib/*" com.yourorg.middleware.QueryDemo medium  
java -cp "lib/*" com.yourorg.middleware.QueryDemo low
```

Output:

- Files: `output/alert_1.json`, `output/alert_2.json`, ...
- Messages: Published to RabbitMQ `alerts_queue`
- Console: Export statistics

Option 2: Multi-threaded Processor

Shell

```
# Compile
javac -cp "lib/*" com/yourorg/middleware/AlertProcessor.java

# Prepare input files
mkdir -p messages
cp /path/to/alerts/*.json messages/

# Run processor
java -cp "lib/*" com.yourorg.middleware.AlertProcessor
```

Behavior:

- Thread 1: Monitors database connection
- Thread 2: Processes files from `messages/` directory
- Thread 3: Consumes messages from RabbitMQ
- Runs until CTRL+C

Option 3: Standalone Listener

Shell

```
# Compile
javac -cp "lib/*" com/yourorg/middleware/RabbitMQListener.java

# Run listener
java -cp "lib/*" com.yourorg.middleware.RabbitMQListener
```

Behavior:

- Connects to RabbitMQ
- Consumes messages continuously
- Inserts to database
- Runs until CTRL+C

8.5 Configuration Management

Hardcoded Values (requires code modification):

- Database: `WazuhAlertDao` lines 13-15

- RabbitMQ: [QueryDemo](#) line 29, [AlertProcessor](#) line 252
- Credentials: Throughout codebase

Recommended Enhancements:

Java

```
// Use environment variables
String dbUrl = System.getenv("DB_URL");
String rabbitHost = System.getenv("RABBITMQ_HOST");

// Or properties file
Properties props = new Properties();
props.load(new FileInputStream("config.properties"));
```

9. Error Handling & Reliability

9.1 Database Error Handling

Connection Failures:

Java

```
try (Connection conn = DriverManager.getConnection(...)) {
    // Operations
} catch (SQLException e) {
    // Connection fails - exception propagates
}
```

Insert Conflicts:

SQL

```
ON CONFLICT (event_id) DO NOTHING
```

- Duplicate event_id: Silently ignored
- Ensures idempotent inserts

Missing/Invalid Data:

- UUID generation for missing event_id
- Current timestamp for invalid/missing timestamp
- Default values for optional fields

9.2 RabbitMQ Error Handling

Connection Failures (QueryDemo):

```
Java
try {
    connection = factory.newConnection();
    // ...
} catch (Exception e) {
    System.out.println("⚠ RabbitMQ not available, continuing
with file export only.");
}
```

- **Graceful degradation:** Continues with file export
- No retry mechanism
- User notified via console

Message Processing Failures:

```
Java
try {
    // Process message
    channel.basicAck(delivery.getEnvelope().getDeliveryTag(),
false);
} catch (Exception e) {
    channel.basicNack(delivery.getEnvelope().getDeliveryTag(),
false, true);
}
```

- **Negative acknowledgment:** Message requeued for retry
- **Requeue=true:** Another consumer or retry attempt can process
- Error logged to console

Consumer Cancellation:

Java

```
channel.basicConsume(QUEUE_NAME, false, deliverCallback,
    consumerTag -> {
        System.out.println("Consumer cancelled: " + consumerTag);
    }
);
```

- Notification of unexpected cancellation
- Application continues running

9.3 File Processing Error Handling

Missing Directory:

Java

```
if (!folder.exists()) {
    folder.mkdirs();
    System.out.println("📁 Place JSON files in messages/
folder");
    return;
}
```

- Auto-creates directory
- Exits gracefully without error

Individual File Failures:

Java

```
try {
    // Process file
    count++;
} catch (Exception e) {
    failed++;
    System.err.println("❌ Failed: " + file.getName());
}
```

- **Continue processing:** One file failure doesn't stop the batch

- Statistics tracked separately
- Error details logged

Invalid JSON:

- JSONException caught and logged
- File skipped, processing continues

9.4 Graceful Shutdown

Shutdown Hook Registration:

Java

```
Runtime.getRuntime().addShutdownHook(new Thread(() -> {
    System.out.println("\n🔴 Shutdown signal received...");
    shutdown();
}));
```

Shutdown Sequence:

1. Stop RabbitMQ listener (close channel/connection)
2. Stop file processor (set running=false)
3. Stop database pool monitor
4. Shutdown executor service
5. Wait up to 10 seconds for thread termination
6. Force shutdown if needed

Resource Cleanup:

- Database connections: Closed via try-with-resources
- RabbitMQ channels: Explicitly closed in finally blocks
- File handles: Closed via try-with-resources
- Threads: Terminated via ExecutorService shutdown

9.5 Retry Mechanisms

Current State:

- **Database:** No automatic retry (connection per operation)
- **RabbitMQ publish:** No retry (fire-and-forget)
- **RabbitMQ consume:** Automatic via NACK with requeue

Recommendations for Enhancement:

Java

```
// Exponential backoff for database
int maxRetries = 3;
for (int i = 0; i < maxRetries; i++) {
    try {
        dao.insertAlert(alert);
        break;
    } catch (SQLException e) {
        if (i == maxRetries - 1) throw e;
        Thread.sleep((long) Math.pow(2, i) * 1000);
    }
}
```

9.6 Monitoring & Observability

Current Logging:

- Console output for all operations
- Emoji indicators for status (✅, ❌, ⚠️, 📁, etc.)
- Statistics reporting

Enhancement Opportunities:

- Structured logging framework (SLF4J + Logback)
- Metrics collection (processed count, error rate)
- Health check endpoints
- Alert on consecutive failures

10. Summary

The Wazuh Alert Processing Middleware provides a flexible, multi-modal approach to security alert management. The architecture supports:

- **Batch processing** via file ingestion
- **Real-time streaming** via RabbitMQ consumers
- **Flexible querying** via severity filters
- **Multi-channel distribution** via simultaneous file and queue outputs

The system's modularity allows deployment in various configurations based on specific requirements, from standalone query tools to fully concurrent processing engines.